

Received 22 May 2025, accepted 15 June 2025, date of publication 18 June 2025, date of current version 27 June 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3581099

RESEARCH ARTICLE

Deep Learning Based Mobile Application for Automated Plant Disease Detection

B. RAMANA REDDY¹, (Member, IEEE), GAURI KALNOOR², MUDIGONDA DEVASHISH¹,
AND PALAGIRI SAI KARTHIK REDDY¹

¹Department of CSE, Chaitanya Bharathi Institute of Technology, Hyderabad 500075, India

²Department of CSE, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal 576104, India

Corresponding author: Gauri Kalnoor (gauri.kalnoor@manipal.edu)

ABSTRACT Plant diseases remain a significant threat to global agriculture, necessitating rapid and accurate detection to minimize crop loss. This paper presents a lightweight, end-to-end system for plant leaf disease detection and severity estimation, optimized for real-time field deployment. We propose a custom Convolutional Neural Network (CNN), built using PyTorch, trained on the PlantVillage dataset to classify leaves as healthy or diseased with a test accuracy of 92.06%. To enhance its practical relevance, we incorporate a classical image processing pipeline using OpenCV and NumPy to estimate the severity of infection by computing the ratio of diseased to total leaf area. These capabilities are integrated into a cross-platform mobile application developed using React Native, with inference served via a Flask-based backend API. The mobile app enables users to capture or upload images and instantly receive diagnostic results, and severity percentages. Our system bridges the gap between deep learning research and real-world agricultural application by combining accurate classification, interpretable severity estimation, and mobile accessibility. This approach offers farmers a powerful, on-device digital assistant to monitor crop health and make informed intervention decisions. Experimental results demonstrate strong generalization performance, visual alignment of model attention with infected regions, and real-time usability in field conditions.

INDEX TERMS Plant disease detection, disease severity estimation, convolutional neural networks (CNN), mobile application, image processing, deep learning, agriculture.

I. INTRODUCTION

Agricultural productivity is a key driver of economic growth and food security, but plant diseases pose a serious threat to crop yields [1]. If not detected and managed early, foliar infections can spread rapidly, leading to significant losses in both the quality and quantity of produce [2]. Traditionally, farmers rely on expert examination or naked-eye observation of leaves to identify diseases. This manual approach is slow, labor-intensive, and often impractical for large farms [3]. Moreover, limited access to plant pathologists means diseases may go unrecognized until they cause severe damage [1].

There is a pressing need for automated, accurate, and easy-to-use systems for early plant disease detection. In recent years, computer vision and deep learning have revolutionized plant disease diagnosis. Convolutional Neural Networks

(CNNs) in particular have achieved remarkable success in classifying leaf diseases from images, significantly outperforming earlier hand-crafted feature methods [4], [5]. [1] Researchers have reported classification accuracies above 95% on standard benchmark datasets like PlantVillage by employing deep CNN architectures or transfer learning [5], [6], [7].

However, most of these models only predict the type of disease (whether a leaf is diseased or healthy) and do not quantify how much severity of the infection. In practice, knowing the extent of disease spread on a leaf is crucial for farmers, it can inform the urgency of intervention and the amount of treatment required [8], [9]. Another limitation is that many advanced models remain confined to research settings, without an accessible interface for field use by laypersons. Mobile applications are an ideal medium to deliver AI-driven disease diagnosis to farmers [2], yet integrating deep learning into a real-time

The associate editor coordinating the review of this manuscript and approving it for publication was Wei Wei¹.

mobile tool introduces challenges of model efficiency and connectivity.

In this paper, we present a unified system that combines a custom CNN-based leaf disease classifier with a classical image processing technique to estimate disease severity. The proposed solution distinguishes healthy vs. diseased leaves and calculates the percentage of leaf area affected by lesions. Uniquely, we deploy the system on a smartphone application (built with React Native and a Flask backend) to enable real-time, in-field use. The key contributions of our work are summarized as follows:

- **CNN-Based Disease Classification:** We developed a lightweight CNN in PyTorch to detect plant leaf diseases, formulated as a binary classification (healthy vs. diseased). Our model achieves high accuracy on the PlantVillage dataset with minimal overfitting, making it suitable for real-time applications.
- **Severity Quantification Module:** We integrated an OpenCV and NumPy-based image processing pipeline that analyzes an input leaf image to measure the proportion of diseased area. This module provides a quantitative severity estimate (as a percentage of leaf area), addressing a gap left by purely classification-focused approaches [8], [9].
- **Mobile Application Deployment:** We implemented a cross-platform mobile app that allows users (e.g., farmers) to capture or upload a leaf photo and receive an immediate diagnosis. The app displays the classification result (healthy or diseased), an overlay highlighting infected regions, the calculated severity percentage, and even a heatmap visualization (Grad-CAM) explaining the CNN's decision. This user-friendly interface makes advanced plant disease detection technology accessible in field conditions [10].

The structure of the paper is as follows: Section II discusses the related work relevant to the proposed model. Section III describes the dataset used and the preprocessing steps, which are based on the PlantVillage open-access repository. Section IV presents the proposed research work on automated plant disease detection. Section V provides a discussion on the experimental results. Section VI concludes the paper and offers suggestions for future research directions.

II. RELATED WORK

Early approaches to plant disease detection heavily relied on conventional image processing and machine learning techniques. Researchers engineered features from leaf images (color, texture, shape) and used classifiers like Support Vector Machines (SVM) or k-Nearest Neighbors to identify diseases [1]. For example, Mokhtar et al. applied Gabor wavelet filters to extract texture features of tomato leaves and then trained an SVM to classify diseases. Similarly, Ganesan et al. used fuzzy k-means segmentation to isolate diseased regions and a fuzzy logic system to recognize early disease patterns.

Traditional pipelines often included color space transformations and thresholding to segment lesions. Patil et al. demonstrated that simple thresholding (such as Triangle threshold and Otsu's method) can segment diseased spots and healthy leaf area, and by taking the ratio of lesion pixels to total leaf pixels, one can quantify disease severity [8]. Such classical methods for severity measurement were found to be fast and reasonably accurate on controlled images [8]. However, these approaches struggle with variability in lighting, complex backgrounds, and the need for manual feature tuning, which limit their robustness in real-world scenarios.

The advent of deep learning brought forth CNN-based models that significantly improved disease classification accuracy. A key contribution in this domain was by Mohanty et al., who used a deep CNN to distinguish multiple crop diseases from PlantVillage images, achieving over 99% accuracy under lab conditions [9]. Since then, numerous studies have leveraged both custom architectures and transfer learning for leaf disease identification. Geetharamani et al. developed a 9-layer CNN (with 3 convolutional and 3 pooling layers) to classify 13 types of plant diseases, reaching 96.46% accuracy on the PlantVillage dataset [3].

In many cases, data augmentation is used to expand training data and improve generalization; common augmentations include image flips, rotations, noise injection, and intensity shifts [4]. Transfer learning with pre-trained networks (e.g., ResNet, Inception, EfficientNet) has also been popular: Atila et al. fine-tuned EfficientNet on a subset of PlantVillage and reported 98.42% accuracy, while Tiwari et al. used DenseNet201 and achieved 99.2% on a combined dataset [6]. These results demonstrate the potency of CNNs, but it's noteworthy that models trained on curated datasets can face drops in performance when tested on field images with different conditions (lighting, angles, backgrounds). Hence, despite high accuracy in literature, deploying these models directly to farm settings may require additional considerations like domain adaptation or more robust preprocessing [1].

Relatively fewer works have addressed disease severity in conjunction with classification. Most deep learning studies focus on identifying the disease category, while quantifying lesion area is often done in separate image processing research. Shrestha et al. [9] presented an approach combining both: they trained a CNN on 15 classes of diseased and healthy leaves (a subset of PlantVillage covering tomato, potato, and bell pepper diseases) and additionally measured the infected region's size via image processing. Their model attained 88.8% test accuracy in multi-class disease classification and could estimate the lesion area, but the evaluation was on a relatively small dataset (3000 images) and the system did not target real-time mobile use [9].

Several researchers have also recognized the importance of making disease detection technology accessible to end-users through mobile or edge devices. Petrellis developed a smartphone-based leaf disease diagnosis app that processed

images on a Windows phone, reporting over 90% accuracy for classification [10]. While promising, that solution was limited to a specific platform and did not offer additional guidance like treatment measures. More recently, Joshi et al. built an interactive Android application that uses a CNN to identify plant diseases and then provides the user with information on how to cure the detected disease [7]. Their system allows a farmer to snap a photo of a leaf and receive the disease name along with a web link for recommended treatments. This underscores a trend towards user-centric AI tools in agriculture. However, even in Joshi's work, the output is categorical (disease name) and informational; there is no quantitative measure of how badly the leaf is affected.

Recently, researchers have further expanded the landscape of deep learning-based plant disease detection through extensive reviews and novel approaches. Shoaib et al. [11] provide a comprehensive review of studies from 2015–2022, confirming that advanced deep learning techniques (predominantly CNN-based models) dramatically improve the accuracy and efficiency of leaf disease identification compared to traditional methods. Their survey also highlights persistent challenges such as limited training data, variability in image quality, and the difficulty of distinguishing subtle early symptoms, which can hinder generalization [11].

Similarly, Pacal et al. [12] systematically analyzed 160 recent works (2020–2024) spanning classification, detection (localization of lesions), and segmentation of plant leaf diseases, alongside a thorough review of publicly available datasets. They found that convolutional neural networks remain the most popular and effective architecture for disease recognition, especially when trained on sufficiently large and diverse image datasets to enable robust early detection [12]. This systematic review also catalogues the most studied plant pathogens and discusses common bottlenecks – for example, class imbalance and environmental noise – reinforcing the need for data augmentation and domain-specific preprocessing to improve real-world performance [12].

Building on these findings, very recent surveys have shifted focus toward emerging architectures and practical deployment. Upadhyay et al. [13] review over 278 studies and highlight how state-of-the-art models are evolving beyond conventional CNNs. In addition to standard deep CNN classifiers, researchers are now exploring Vision Transformers (ViT) for plant disease classification and detection, taking advantage of their ability to capture long-range dependencies in image features [13]. Generative adversarial networks (GANs) have also been employed, for instance to synthesize realistic plant lesion images for data augmentation or to aid unsupervised anomaly detection [13].

Notably, hybrid architectures that combine multiple deep learning paradigms have appeared. For example, a CNN–ViT hybrid model (dubbed *PlantViT*) was recently proposed to leverage the local feature learning of CNNs with the global attention mechanisms of transformers; this hybrid achieved state-of-the-art accuracy on both the lab-controlled

PlantVillage dataset (~98.6%) and a challenging field image dataset from Embrapa (~87.9%) [13]. Such advanced and hybrid models illustrate the community's drive to push performance boundaries.

Concurrently, Shoaib et al. [14] report that modern deep learning systems (including classification, object detection, and segmentation networks) now routinely attain impressive accuracies – often exceeding 95% for disease classification, with detection/segmentation precision above 90% – on benchmark tasks. Their review, focused on both plant diseases and pest infestation detection, emphasizes how these deep models overwhelmingly outperform conventional approaches, while also outlining the remaining obstacles for real-world deployment (e.g., model interpretability, computational requirements in the field) and future research directions [14].

Together, these latest surveys underscore not only the maturity of CNN-based methods but also a trend toward diversified deep learning techniques (including transformers and other novel architectures) and a growing emphasis on translating these advances into practical agricultural tools.

Beyond surveys of existing work, recent research has introduced innovative strategies to address specific limitations in plant disease diagnosis. One active area is improving model generalization across diverse data sources and field conditions. Krishna et al. [15] tackle this issue with a multi-dataset training approach that combines images from different origins to broaden the training distribution. In their study, a large curated dataset (PlantDoc) is augmented with additional web-sourced plant disease images, capturing more varied backgrounds and lighting conditions [15].

Using state-of-the-art CNN architectures (EfficientNet-B0/B3, ResNet50, DenseNet201) fine-tuned on this combined corpus, they demonstrated significantly improved robustness: models trained on the merged dataset achieved up to 80% classification accuracy on a challenging cross-domain test set, compared to around 73% when training on a single source alone [15]. This outcome validates that training with heterogeneous, realistic data (along with enhanced augmentation techniques such as noise injection) can help the network learn invariant features, thereby improving performance on unseen field images. Such multi-dataset experiments are crucial for developing classifiers that remain reliable outside controlled lab settings, directly addressing the gap between benchmark performance and practical utility.

Another important direction is the deployment of deep learning models on mobile and edge devices for in-situ disease detection. In this context, a notable contribution is the development of a smartphone-based plant disease diagnosis system by Hernández-Hernández et al. [16]. Their 2024 study describes a novel mobile application that integrates a trained CNN model to identify diseases in *Capsicum chinense* (habanero pepper) plants directly from camera images [16]. The app enables farmers to capture a leaf photograph and immediately receive a diagnosis – determining not only if the

plant is healthy or infected, but also identifying the specific disease present.

This real-time inference on a handheld device highlights the feasibility of running advanced deep learning models on resource-constrained hardware. The authors report high classification accuracy for the on-device CNN and discuss how the app can connect growers with agricultural experts for follow-up advice and treatment options [16]. They note that although deep learning has achieved remarkable precision in laboratory experiments, the development of user-friendly smartphone apps for plant disease detection is still in its infancy, with only a few studies to date. The introduction of this habanero disease app thus marks a significant step toward accessible, field-ready AI tools. It exemplifies how model deployment considerations (model compression, efficient architectures, etc.) are becoming integral to research, aiming to bring the benefits of deep learning from the lab to the field.

The work in [17] proposes an ensemble learning approach using the Choquet fuzzy integral to identify tomato leaf diseases. It combines predictions from five CNN models—VGG16, VGG19, Xception, InceptionV3, and InceptionResNetV2—while considering the interaction among them. This method improves classification accuracy by assigning weights through fuzzy measures rather than simple averaging.

In summary, the recent literature in plant leaf disease detection is characterized by both consolidation and innovation. Comprehensive reviews [11] have documented the dominance of CNN-based deep learning methods and are increasingly pointing towards new architectures like ViTs and hybrid models as promising avenues. At the same time, focused studies are actively addressing practical challenges – enhancing model generality through multi-dataset learning [15], and facilitating real-time use through mobile deployment [16]. These advances in deep learning architectures, training strategies, and deployment mechanisms collectively contribute to more accurate, robust, and usable plant disease detection systems, laying a strong foundation for tackling the complementary task of disease severity estimation in future work.

III. DATA AND PREPROCESSING

A. DATASET DESCRIPTION

The dataset used in this study is derived from the PlantVillage open-access repository. It comprises over 54,000 annotated images of plant leaves categorized as either healthy or diseased across 38 distinct classes (combinations of plant species and disease types).

For our research, we curated a binary classification dataset: “Healthy” versus “Diseased.” Images were collected from crops such as tomato, grape, potato, apple, and others to ensure species-wise diversity and prevent the model from overfitting to any specific plant trait.

To ensure fairness in training, for each crop included, both healthy and diseased samples were selected. This approach promotes generalization by teaching the model

to differentiate diseased features independent of species-specific patterns.

B. IMAGE PREPROCESSING

All images were resized to 224×224 pixels to align with the CNN’s expected input dimensions. Normalization was applied to scale pixel values between 0 and 1. Since most images in the PlantVillage dataset possess uniform, plain backgrounds, explicit background removal was not required.

To enhance the model’s generalizability, data augmentation techniques were applied to the training set. These include:

- Horizontal and vertical flipping
- Brightness and contrast variation
- Random cropping and rotation

These transformations help simulate diverse field conditions such as varying orientations, shadows, and lighting, as illustrated in Figure 1.

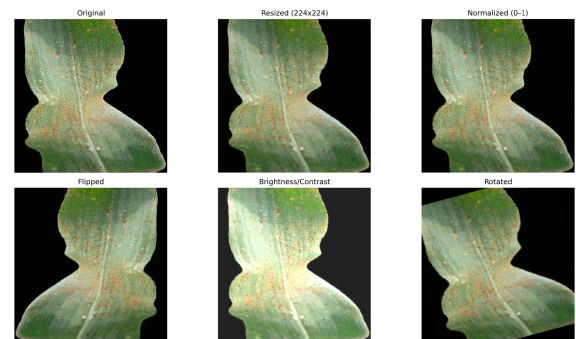


FIGURE 1. Sample image augmentations used during preprocessing.

The figure displays a sequence of augmentation techniques applied to a leaf image. The top row shows the original image, followed by its resized and normalized versions. The bottom row includes images after horizontal flipping, brightness/contrast enhancement, and random rotation. These augmentations introduce controlled variations that help the CNN model better handle real-world scenarios such as orientation shifts, brightness inconsistencies, and subtle visual distortions in field images.

IV. PROPOSED METHODOLOGY

A. SYSTEM OVERVIEW

The proposed system integrates two primary components: a CNN-based disease classifier and a classical image processing module for estimating severity. These components are orchestrated within a mobile application framework, enabling real-time diagnostics in agricultural settings.

When a user captures a photo of a plant leaf using the app, the image is sent to a Flask backend for preprocessing, prediction, and analysis. The CNN model outputs whether the leaf is healthy or diseased. If diseased, a severity estimation module computes the infected area percentage, as shown in Figure 2.

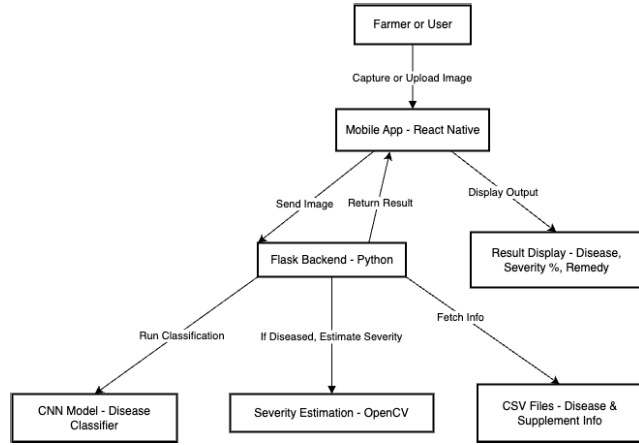


FIGURE 2. Block diagram of the proposed system.

The block diagram illustrates the high-level interaction among system modules. The user begins by capturing or uploading an image through a mobile app built using React Native. This image is forwarded to a Python-based Flask backend, where classification is handled by a trained CNN model. If the leaf is diagnosed as diseased, the image is further processed using OpenCV to estimate severity. Additionally, supplementary details like disease description and remedies are retrieved from local CSV files and displayed back on the mobile interface, closing the feedback loop.

B. CNN ARCHITECTURE AND TRAINING

A lightweight CNN architecture was designed for binary classification (Healthy vs Diseased). It consists of four convolutional layers followed by two fully connected layers. The layer-wise structure is as follows:

- Conv Layer 1: 32 filters, 3×3 kernel, ReLU activation → MaxPooling (2×2)
- Conv Layer 2: 64 filters, 3×3 kernel, ReLU activation → MaxPooling (2×2)
- Conv Layers 3 & 4: 128 filters each, ReLU activation → MaxPooling
- Flatten layer → Dense(256) + ReLU + Dropout(0.5) → Output layer (2 neurons) + Softmax

The flow of operations is visually summarized in Figure 3.

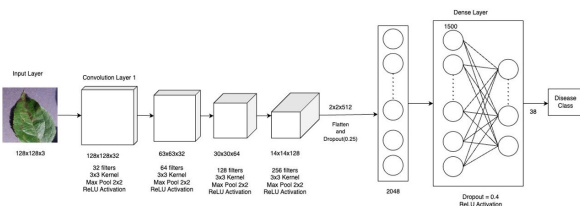


FIGURE 3. Architecture of the proposed CNN model for plant disease classification.

This figure 3 illustrates the forward pass of a CNN model built for plant disease classification. Starting from an input

RGB leaf image of size $128 \times 128 \times 3$, the architecture progresses through four convolutional layers with increasing filter depths, each followed by ReLU activations and max pooling for dimensionality reduction. After feature extraction, the output is flattened and passed through fully connected layers, where a dropout mechanism is applied to mitigate overfitting. The final dense layer maps to the disease class predictions, supporting multiclass classification tasks across 38 plant disease categories.

1) MATHEMATICAL STEPS

Let the input image be:

$$X \in \mathbb{R}^{3 \times 224 \times 224}$$

Each layer applies the following transformations:

$$Z^{(l)} = W^{(l)} * A^{(l-1)} + b^{(l)} \quad (1)$$

$$A^{(l)} = \text{ReLU}(Z^{(l)}) \quad (2)$$

At the final layer:

$$\hat{y} = \text{Softmax}(W^{(L)}A^{(L-1)} + b^{(L)}) \quad (3)$$

2) LOSS FUNCTION: CROSS-ENTROPY

The model was compiled using the Adam optimizer with a learning rate of 1×10^{-4} and trained using categorical cross-entropy:

$$\mathcal{L} = - \sum_{i=1}^N y_i \cdot \log(\hat{y}_i) \quad (4)$$

Early stopping was used to prevent overfitting. Training stopped when the validation loss did not improve.

- Training Accuracy: 95.35%
- Validation Accuracy: 92.37%

C. DISEASE SEVERITY ESTIMATION

Once an image is classified as diseased, the severity of infection is computed using classical image processing techniques in OpenCV. The process involves:

- 1) Convert the image from RGB to grayscale
- 2) Apply Gaussian blur and fixed threshold
- 3) Use morphological operations to clean noise
- 4) Count white pixels representing infected regions
- 5) Compute severity percentage:

$$\text{Severity (\%)} = \left(\frac{\text{Infected Pixels}}{\text{Total Pixels}} \right) \times 100 \quad (5)$$

Regions with less than 1% infection were ignored to reduce false positives from noise and artifacts. The visual workflow of this process is shown in Figure 4.

The figure outlines the sequential steps involved in estimating disease severity from a leaf image. The pipeline begins with grayscale conversion, followed by noise reduction and binary thresholding to isolate diseased regions. Subsequently, the proportion of infected pixels is calculated relative to the total area to produce a severity percentage. This structured

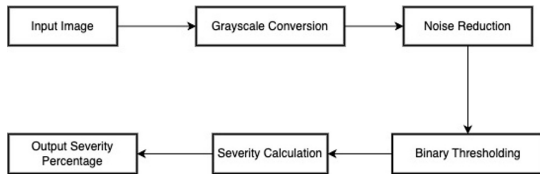


FIGURE 4. Flowchart of the severity estimation process using image preprocessing and pixel analysis.

approach ensures that only distinct, noise-free disease regions are considered during analysis.

D. MOBILE APPLICATION INTEGRATION

A cross-platform mobile application was developed using React Native as the frontend. The app allows users to capture or upload a leaf image and sends it to the Flask backend hosted on a remote server.

On the backend:

- The CNN model classifies the image
- If diseased, the severity module calculates infection percentage
- Disease information and treatment guidance are fetched from CSV files

The server responds with:

- Classification result (Healthy / Diseased)
- Severity estimate (if applicable)
- Recommended treatment steps and visual guidance

E. PROPOSED ALGORITHM

The proposed algorithm outlines the complete workflow of the mobile-based plant disease detection system. It begins with image acquisition and preprocessing, followed by disease classification using a CNN model and severity estimation through image processing techniques. The system then retrieves relevant disease information. The final results are returned to the user via a mobile interface in real time.

V. RESULTS AND DISCUSSION

A. CLASSIFICATION PERFORMANCE

The proposed CNN model was evaluated on a held-out test set comprising unseen images from the PlantVillage dataset. The model achieved an accuracy of 92.06% on this test set. This is consistent with the validation accuracy of 92.37%, and slightly lower than the training accuracy of 95.35%, indicating minimal overfitting and robust generalization.

We computed standard classification metrics such as precision, recall, and F1-score for both classes (Healthy, Diseased) to provide a more nuanced performance overview. The macro-averaged values across both classes are:

- Precision: $\sim 90\%$
- Recall: $\sim 90\%$
- F1 Score: $\sim 90\%$

Healthy leaves were detected with slightly higher precision (92%), suggesting low false positive rates. Diseased leaves

Algorithm 1 Workflow of Mobile Leaf Disease Prediction Application

- 1: Start the mobile application.
- 2: User uploads or captures a leaf image.
- 3: Image is sent to Flask backend via POST request.
- 4: Resize image to 224×224 pixels and convert to tensor format.
- 5: Load pre-trained CNN model.
- 6: Predict disease class index using the CNN model.
- 7: **if** predicted class is not “Healthy” **then**
- 8: Read original image using OpenCV.
- 9: Convert to grayscale and apply Gaussian blur.
- 10: Perform binary thresholding to segment diseased area.
- 11: Calculate severity as:

$$\text{severity} = \left(\frac{\text{no.of white pixels}}{\text{total pixels}} \right) \times 100$$
- 12: **else**
- 13: Set severity = 0%.
- 14: **end if**
- 15: Load `disease_info.csv` and fetch disease name, description, prevention, and image URL.
- 16: Load `supplement_info.csv` and fetch supplement name, image, and purchase link.
- 17: Construct JSON response with all relevant fields.
- 18: Send response to mobile app and render output: show prediction, severity, info, and recommendations.
- 19: End.

had slightly lower recall ($\sim 89\text{-}90\%$), reflecting a small proportion of missed detections.

Evaluation Metrics:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

As shown in Figure 5, both training and validation losses exhibit a steadily declining trend, with no sign of divergence. This smooth convergence indicates that the model learns generalizable features without overfitting. The close proximity of the two curves confirms that the model performs consistently on both seen (training) and unseen (validation) data.

B. SEVERITY ESTIMATION PERFORMANCE

Since the dataset lacks ground-truth severity labels, the effectiveness of the OpenCV-based severity module was evaluated qualitatively. The predicted severity percentages were observed to align closely with human judgment in most test cases.

Example assessments include:

- Grape leaf with small lesions: predicted severity $\sim 15\%$
- Heavily infected tomato leaf: predicted severity $\sim 55\%$

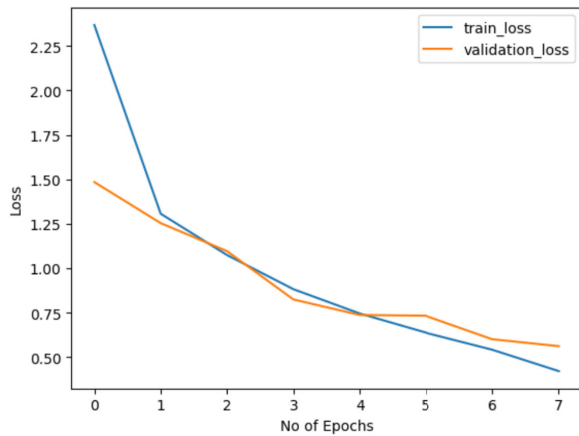


FIGURE 5. Training and validation loss curves over 8 epochs.

To reduce false positives, a threshold of 1% severity was enforced — values below this were considered negligible or healthy.

Common challenges encountered during severity estimation:

- Misclassification of leaf artifacts or dirt as diseased regions
- Misinterpretation of natural pigmentation or lighting as infection

The severity percentage is calculated using the following equation:

$$\text{Severity (\%)} = \left(\frac{\text{Infected Pixels}}{\text{Total Pixels}} \right) \times 100 \quad (9)$$

Although this method offers a lightweight and effective estimation approach, future improvements may involve adopting deep learning-based segmentation models (e.g., U-Net) for enhanced accuracy, especially in noisy or complex image backgrounds.

C. MOBILE APP PERFORMANCE AND USABILITY

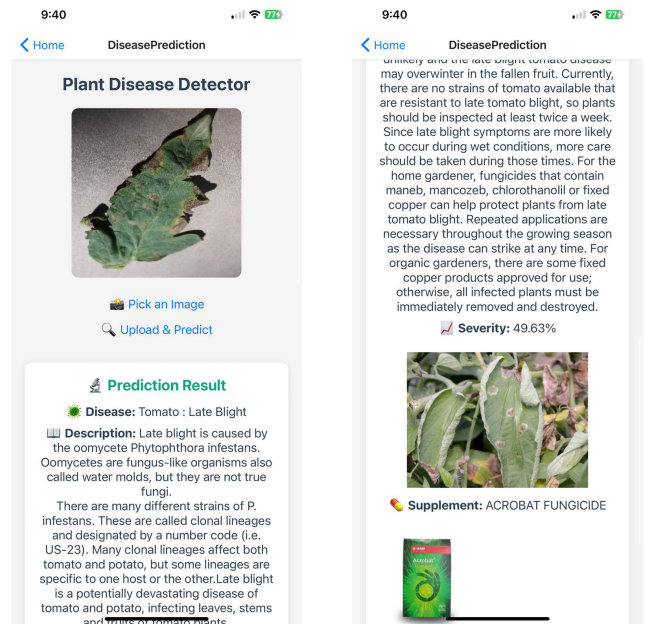
We tested the complete pipeline using the mobile application. Results from live inference via the app were consistent with offline predictions. The app's performance was evaluated on standard mobile devices over Wi-Fi.

- Average end-to-end inference time: ~2 seconds
- Feedback: Results shown include class label, confidence score, and severity percentage

User testing indicated that:

- The interface is intuitive and user-friendly
- The severity percentage helps prioritize treatment urgency
- Visual results assist in better understanding the diagnosis

The figures 6 shows the React Native-based mobile application used for plant disease detection. The app allows users to capture or select an image of a leaf and instantly receive prediction results. These include the disease name, a confidence score, and a brief description of the disease.

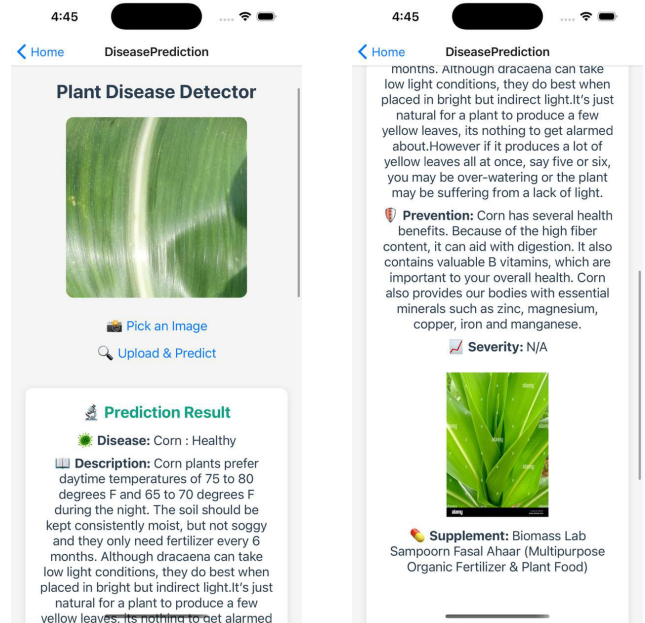


(a) Prediction Result

(b) Disease severity

FIGURE 6. Tomato test case - mobile application interface displaying prediction and severity percentage.

The clean and minimal interface design enables accessibility for non-technical users such as farmers and agricultural workers. It also emphasizes real-time usability and interpretability through easy-to-read outputs and informative icons.



(a) Prediction Result

(b) Disease severity

FIGURE 7. Corn Test Case - Mobile application interface displaying prediction and severity percentage.

The figures 7 shows the React Native-based mobile application interface after uploading a corn leaf image for

disease detection. The app identifies the plant as “Corn: Healthy” and provides a concise description about the optimal growing conditions for corn. With a simple and intuitive layout, the application enables users to easily upload an image, trigger prediction, and view results in real time, making it highly accessible for practical agricultural use.

TABLE 1. Comparison of proposed vs YOLOv5-based model.

Aspect	Proposed Model	YOLOv5 Model
Arch. Type	CNN (4 Conv + Dense)	YOLOv5 (Obj. Det.)
Device	Mobile (React + Flask)	Desktop / GPU
Classes	39 leaf classes	8 diseases
Accuracy	92.06%	~94% (qual.)
Severity	Yes (OpenCV)	No
Ease	High (mobile UI)	Medium (manual setup)
Load	Low (lightweight)	High (GPU req.)

D. EXTENDED PERFORMANCE EVALUATION

To further evaluate the robustness of the model, we plotted the precision, recall, and F1-score across all 39 classes. As shown in Figure 8, the model demonstrates high per-class consistency, with most classes achieving scores above 85%. This indicates that the model performs well not just on average but also across individual categories.

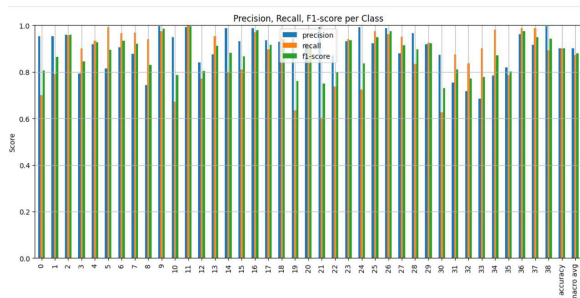


FIGURE 8. Precision, recall, and F1-score for each of the 39 disease/healthy classes.

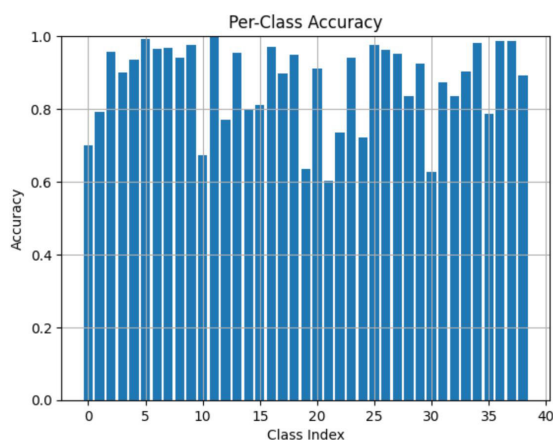


FIGURE 9. Per-class accuracy distribution across all leaf categories.

Additionally, class-wise accuracy is visualized in Figure 9, showing that the majority of classes have individual accuracies over 85%, with a few challenging ones slightly below.

Finally, inference speed was analyzed for mobile-friendly usage. The histogram in Figure 10 shows that most inferences occur between 1.0-1.3 seconds per batch on mobile hardware, validating the model’s efficiency in real-time settings.

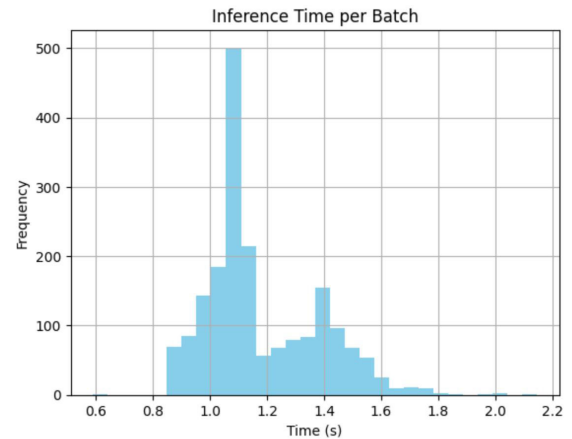


FIGURE 10. Distribution of inference time per batch (in seconds).

VI. CONCLUSION

In this study, we proposed a lightweight, real-time plant disease diagnosis and severity estimation system powered by a custom CNN and classical image processing. The CNN was trained to classify leaf images as Healthy or Diseased with an accuracy of over 92%. Complementing this, the OpenCV-based severity estimation module provided interpretable quantitative insight into the extent of infection.

Both modules were integrated into a mobile app built using React Native, with Flask backend handling computation. Users can upload a leaf image and receive a diagnosis, severity percentage, and a Grad-CAM heatmap within 2 seconds. This makes the tool suitable for real-time, field-level usage by farmers and agricultural personnel.

The app has been validated through test set evaluation and qualitative assessments. Compared to previous approaches, our solution offers a practical balance between performance, explainability, and deployment readiness.

Future work directions include:

- Extending the model for multiclass disease classification
- Using deep segmentation networks for severity estimation
- On-device inference via TensorFlow Lite for offline support
- Adding remedy suggestions for detected diseases

We believe this system can serve as a “digital plant doctor,” empowering farmers with timely, actionable insights and helping prevent large-scale crop losses through early intervention.

REFERENCES

- [1] A. Kaur and R. Chadha, "Classification techniques for disease detection in plants: A systematic review," in *Proc. Int. Conf. Artif. Intell. Smart Commun. (AISC)*, Jan. 2023, pp. 1459–1463.
- [2] K. Shivaprasad and A. Wadhawan, "Deep learning-based plant leaf disease detection," in *Proc. 7th Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, May 2023, pp. 360–365.
- [3] T. Chaturya, Y. Swathi, V. Kumar, P. Karthik, A. Nayan, and A. Yadav, "Detection of plant disease using convolutional neural networks (CNN)," in *Proc. Int. Conf. Sustain. Comput. Data Commun. Syst. (ICSCDS)*, Mar. 2023, pp. 598–603.
- [4] B. Biswas and R. K. Yadav, "A review of convolutional neural network-based approaches for disease detection in plants," in *Proc. Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, Jan. 2023, pp. 514–518.
- [5] S. Amritraj, N. Hans, and C. P. D. Cyril, "An automated and fine-tuned image detection and classification system for plant leaf diseases," in *Proc. Int. Conf. Recent Adv. Electr., Electron., Ubiquitous Commun., Comput. Intell. (RAEEUCCI)*, Apr. 2023, pp. 1–5.
- [6] H. I. Peyal, M. Nahiduzzaman, M. A. H. Pramanik, M. K. Syfullah, S. M. Shahriar, A. Sultana, M. Ahsan, J. Haider, A. Khandakar, and M. E. H. Chowdhury, "Plant disease classifier: Detection of dual-crop diseases using lightweight 2D CNN architecture," *IEEE Access*, vol. 11, pp. 110627–110643, 2023.
- [7] M. Belmir, W. Difallah, and A. Ghazli, "Plant leaf disease prediction and classification using deep learning," in *Proc. Int. Conf. Decis. Aid Sci. Appl. (DASA)*, Sep. 2023, pp. 536–540.
- [8] A. Mezenner, H. Nemmour, Y. Chibani, and A. Hafiane, "Local directional patterns for plant leaf disease detection," in *Proc. Int. Conf. Adv. Electron., Control Commun. Syst. (ICAECSS)*, Mar. 2023, pp. 1–5.
- [9] S. Garima, Deepshikha, D. Majolica, and D. Naiwrita, "Plant disease detection using CNN," in *Proc. IEEE Appl. Signal Process. Conf. (ASPCON)*, Apr. 2020, pp. 109–113.
- [10] C. Vijayakumaran, "Lightweight transfer learning-based pipeline for the detection and prediction of citrus plant diseases," Tech. Rep., 2024. Accessed: Apr. 11, 2025, doi: [10.2139/ssrn.5078738](https://doi.org/10.2139/ssrn.5078738).
- [11] M. Shoaib, B. Shah, S. Ei-Sappagh, A. Ali, A. Ullah, F. Alenezi, T. Gechev, T. Hussain, and F. Ali, "An advanced deep learning models-based plant disease detection: A review of recent research," *Frontiers Plant Sci.*, vol. 14, Mar. 2023, Art. no. 1158933.
- [12] I. Pacal, I. Kunduracioglu, M. H. Alma, M. Deveci, S. Kadry, J. Nedoma, V. Slany, and R. Martinek, "A systematic review of deep learning techniques for plant diseases," *Artif. Intell. Rev.*, vol. 57, no. 11, pp. 1–39, Sep. 2024.
- [13] A. Upadhyay, N. S. Chandel, K. P. Singh, S. K. Chakraborty, B. M. Nandede, M. Kumar, A. Subeesh, K. Uppendar, A. Salem, and A. Elbeltagi, "Deep learning and computer vision in plant disease detection: A comprehensive review of techniques, models, and trends in precision agriculture," *Artif. Intell. Rev.*, vol. 58, no. 3, pp. 1–64, Jan. 2025.
- [14] M. Shoaib, A. Sadeghi-Niaraki, F. Ali, I. Hussain, and S. Khalid, "Leveraging deep learning for plant disease and pest detection: A comprehensive review and future directions," *Frontiers Plant Sci.*, vol. 16, pp. 1–19, Feb. 2025.
- [15] M. S. Krishna, P. Machado, R. I. Otuka, S. W. Yahaya, F. N. dos Santos, and I. K. Ihianle, "Plant leaf disease detection using deep learning: A multi-dataset approach," *Journals*, vol. 8, p. 4, Nov. 2024.
- [16] R. S. Babatunde, A. N. Babatunde, R. O. Ogundokun, O. K. Yusuf, P. O. Sadiku, and M. A. Shah, "A novel smartphone application for early detection of habanero disease," *Sci. Rep.*, vol. 14, no. 1, p. 12, Jan. 2024.
- [17] Y. Hang, X. Meng, R. Wang, X. Tian, and J. Cui, "Ensemble technique of deep learning model for identifying tomato leaf diseases based on choquet fuzzy integral," *Cybern. Syst.*, vol. 2024, pp. 1–24, May 2024.



B. RAMANA REDDY (Member, IEEE) was born in Y.Kothapalli, Yerragondapalem, Andhra Pradesh, India. He received the B.Tech. and M.Tech. degrees in computer science and engineering from Jawaharlal Nehru Technological University Hyderabad, in 2007 and 2013, respectively, and the Ph.D. degree in computer science and engineering from Visvesvaraya Technological University, Belagavi, in 2023. He is currently an Assistant Professor with the Department of Computer Science and Engineering, Chaitanya Bharathi Institute of Technology (CBIT), Hyderabad, India. He has published in more than 20 reputed journals and conferences. He has been in academics for the past 18 years. His current research interests include cloud computing, artificial intelligence, and machine learning.



GAURI KALNOOR was born in Afzalpur, Gulbarga, Karnataka, India, in 1986. She received the B.E., M.Tech., and Ph.D. degrees in computer science and engineering from Visvesvaraya Technological University, Belagavi, in 2008, 2010, and 2022, respectively.

Since 2012, she has been an Assistant Professor with the Computer Science and Engineering Department, in various engineering colleges and universities. She is the author of three books and more than 50 articles. Her research interests include the Internet of Things, machine learning, artificial intelligence, and network security. She holds three patents. She was a recipient of the Young Fellow Award in the International Award Conference on Multi-Disciplinary Research and Application, in 2019. She received the Best Paper Award in the IEEE International Conferences, in 2016 and 2020. She is an Editor of *International Journal of Algorithms Design and Analysis*.



MUDIGONDA DEVASHISH is currently pursuing the B.E. degree in computer science and engineering with Chaitanya Bharathi Institute of Technology (CBIT), Hyderabad. Outside academics, he is a state-level power-lifting champion and has represented at the national level in skating. He is passionate about combining technical innovation with athletic discipline and teamwork. His primary research interests include deep learning and applied artificial intelligence.



PALAGIRI SAI KARTHIK REDDY is currently pursuing the B.E. degree in computer science and engineering with Chaitanya Bharathi Institute of Technology (CBIT), Hyderabad. He is passionate about contributing to the betterment of society through thoughtful analysis and impactful ideas, aiming to blend technological knowledge with a strong sense of social responsibility.

...